MNIST Presentation

Slide 1

A warm welcome to my presentation on the design and evaluation of an artificial neural network for classifying handwritten digits.

The aim of this project was to develop a machine learning model that could recognise handwritten digits from zero to nine using the MNIST dataset. I will explain how I prepared the data, designed and trained the network, evaluated its performance, and compared models with and without dropout regularisation.

I will also discuss the limitations of my approach and identify improvements that could be explored in future work.

Slide 2

The aim was to develop and evaluate an artificial neural network for handwritten digit classification.

I divided this into several objectives. First, I prepared the dataset and separated the supplied training data into training and validation sets. I then designed and trained a neural network to classify the images. I subsequently evaluated its performance using accuracy, loss and a confusion matrix.

Finally, I compared two configurations, one with dropout and one without, to investigate whether dropout affected model performance.

In approaching it this way, it allowed me to consider not only whether the model could make accurate predictions, but also whether its performance was consistent and what the results revealed about the design.

Slide 3

The dataset used was MNIST, a widely used dataset for handwritten digit recognition and has become a standard for evaluating classification approaches. It contains 70,000 greyscale images representing the digits zero to nine (Lecun et al., 1998). Each image is 28 by 28 pixels, with an associated label identifying the correct digit (Keras).

The dataset is divided into 60,000 images supplied for training and 10,000 separate images for testing. The relatively small image dimensions make the dataset suitable for experimenting with neural network design and evaluation.

However, the fact that MNIST is a standardised dataset is also a limitation. A person's everyday handwriting can vary considerably in writing style, image quality, orientation and background. This can therefore mean high performance on MNIST does not automatically demonstrate that the model would perform equally well in a real world setting.

Slide 4

I divided the 60,000 images supplied as training data into 54,000 training images and 6,000 validation images. I retained the separate 10,000 image test set for final evaluation.

These three sets have different purposes. The training set was used to teach the network by adjusting its internal parameters. The validation set allows performance to be monitored during development and helps assess whether the model is generalising beyond the images it learns from. The test set provides a separate evaluation after the model has been developed.

I used a stratified split so that the proportions of the ten digit classes were kept broadly consistent across the training and validation sets.

Maintaining a distinction between model development and final evaluation is important as repeated use of evaluation data during model selection can produce overly optimistic estimates of performance (Cawley & Talbot, 2010). The validation set provides additional evidence during development, while the independent test set helps assess performance on unseen examples.

Slide 5

Before training, I prepared the image data so that it could be used consistently by the network.

Each image was represented as a 28x28 array of pixel values. Originally, the pixel values ranged from 0 to 255. I normalised these values by dividing them by 255, converting them to a range between 0 and 1.

This is useful because it places the input values on a smaller, consistent scale, which helps the neural network training behave more effectively.

I applied the same normalisation to the training, validation and test images. The digit labels remained as integers from 0 to 9 because the model was designed to predict 1 of 10 classes.

Normalising the images does not change which digit an image represents but changes the numerical scale of the image data so that it is more suitable for training.

Slide 6

I designed a fully connected artificial neural network using an input layer, two hidden dense layers and an output layer.

The input consists of a 28x28 image. The flatten layer converts this two-dimensional image into a one-dimensional vector containing 784 pixel values. This allows the dense layers to process the image as a list of numerical inputs.

The first dense layer contains 128 neurons, followed by a second dense layer containing 64 neurons and both use the ReLU activation function. ReLU introduces non-linearity, allowing

the network to learn more complex relationships between the input pixels and the digit classes. ReLU activation functions have been investigated as an effective approach for training neural networks, including their ability to learn useful feature representation (Glorot et al., 2010)

I included a dropout layer with a rate of 0.2 between the hidden layers. During training, dropout randomly disables a proportion of neurons, which discourages the network from relying too heavily on particular neurons. It is a regularisation technique intended to help reduce overfitting (Srivastava et al., 2014).

The final layer contains ten neurons, one for each possible digit. I used softmax activation to produce a probability distribution across the ten classes. The predicted digit is the class with the highest predicted probability.

I used Adam as the optimiser. An optimiser updates the network's internal parameters during training in an attempt to reduce prediction errors. Adam was selected as a practical optimisation method commonly used in neural network training and uses adaptive estimates of gradient information to update model parameters during optimisation (Kingma et al., 2014).

The loss function measures how well the predicted class probabilities match the correct digit labels. The word 'sparse' refers to using integer class labels rather than one hot encoded labels.

The model was trained for up to ten epochs, with a batch size of 128. An epoch represents one complete pass through the training data, while the batch size determines how many images are processed together before an update is made.

These choices provided a manageable baseline model that could be trained and evaluated within the scope of the project.

Slide 7

I assessed the learning process using training and validation accuracy and loss across the ten epochs.

Accuracy measures the proportion of predictions that are correct. Loss measures the model's prediction error according to the selected loss function and consideration of both measures gives a more informative picture than relying on accuracy alone.

For the model with dropout, the final training accuracy was approximately 98.36%, while the final validation accuracy was approximately 97.42%. The best recorded validation accuracy was approximately 97.77%.

The learning curves showed that the model learned useful patterns from the training data. However, training performance was higher than validation performance and this gap suggests that some overfitting may be present: the model may fit the training data better than it generalises to unseen examples.

The gap does not, by itself, prove serious overfitting. It needs to be interpreted alongside the loss curves and the final test results.

Slide 8

I evaluated the model on the separate test set of 10,000 images. It achieved a test accuracy of 97.86%, correctly classifying 9,786 images and misclassifying 214.

This indicates strong performance on the MNIST test data but overall accuracy does not explain which digits the model finds most difficult to recognise.

For this reason, I also examined the confusion matrix. A confusion matrix compares the actual classes with the classes predicted by the model. Correct predictions appear along the main diagonal, while off diagonal entries show errors and the classes involved.

The matrix showed that some digits were more frequently confused with others, for example, the results included cases where an actual five was predicted as a three, and an actual four was predicted as a nine.

These errors demonstrate that the model does not recognise every handwritten digit correctly. Similar looking handwriting can create ambiguity, even when the model performs well overall.

The confusion matrix therefore provides a more detailed evaluation of model behaviour than accuracy alone. It can also help identify which classes may need further investigation.

Slide 9

To investigate the effect of dropout, I compared the model with dropout against a configuration without dropout.

The model with dropout achieved a final training accuracy of 98.36%, validation accuracy of 97.42%, and validation loss of 0.0899. The model without dropout achieved a training accuracy of 99.43%, validation accuracy of 97.45%, and validation loss of 0.0933.

The model without dropout had slightly higher final validation accuracy, with a difference of approximately 0.03 percentage points, but also had a higher validation loss and a larger gap between training and validation accuracy.

This comparison illustrates why model selection should not rely on accuracy alone. The model without dropout classified a slightly greater proportion of validation images correctly in this run, but its other metrics do not establish that it is definitively better.

Equally, the results do not prove that dropout improved generalisation. I conducted one comparison of these configurations, so the small difference may not be consistent across repeated runs. Repeating the experiments with controlled random seeds would help establish whether the observed differences are reproducible.

This is an important limitation of the comparison and a reason to interpret the findings cautiously.

Slide 10

Overall, the model achieved 97.86% test accuracy, demonstrating that a relatively straightforward fully connected neural network can classify the standardised MNIST images effectively.

There are however a few limitations to consider, namely, the difference between training and validation performance suggests that the model may be overfitting to some extent. Dropout was included to help address this, but the comparison did not demonstrate a clear improvement in validation accuracy. Further controlled experiments would be needed to reach a stronger conclusion.

Second, the flatten layer converts each image into a one dimensional vector. Although this allows the dense layers to learn patterns from the pixel values, it removes the two dimensional arrangement of the image from the representation. A convolutional neural network, or CNN, could be investigated as an alternative because convolutional layers are designed to learn local spatial patterns in images. CNN's were developed to exploit spatial structure in images and have been successfully applied to handwritten character recognition (LeCun et al., 1998).

MNIST also contains relatively small, standardised greyscale images. Regular handwriting may include different backgrounds, image quality, orientations and writing styles, which means the model's test accuracy should not be interpreted as evidence that it is ready for use in an unrestricted application.

Possible improvements include comparing the ANN with a CNN, repeating training runs using different random seeds, and examining the misclassified images to identify recurring patterns. Additional evaluation metrics, such as precision, recall and F1-score for each digit class, could also give a more detailed view of classification performance.

Slide 11

By the end of this assignment, I developed and evaluated an artificial neural network to classify handwritten digits from 0 to 9 using the MNIST dataset.

The dataset was prepared through normalisation and a separate training and validation split. I designed a network with two hidden dense layers, ReLU activation, a softmax output layer and dropout regularisation.

The model achieved 97.86% accuracy on the separate test set, and I also used learning curves and a confusion matrix to investigate its performance and identify classification errors.

The comparison between models with and without dropout showed that slightly higher accuracy does not necessarily mean a model is definitively better. Training and validation gaps, loss and reproducibility also need to be considered.

From this I recognised importance of evaluating a machine learning model critically, recognising its limitations and distinguishing measured results from proposed future improvements.

Google colab notebook

I will now briefly demonstrate the implementation in Google Colab.

First, I will show the code used to load the MNIST dataset and inspect the image dimensions and digit labels. This demonstrates the data used in the project.

Next, I will show the preprocessing and the separation of the supplied training data into training and validation sets. I will then point out the model architecture, including the hidden dense layers, dropout layer and 10 neuron output layer.

Finally, I will show the evaluation results, including the test accuracy and confusion matrix. These outputs provide evidence of how the model performed and illustrate the difference between a summary metric and a more detailed analysis of classification errors.

The demonstration links the design choices discussed in the presentation to the actual implementation and evaluation. It also shows how the results can be inspected rather than relying only on the reported accuracy.

Thank you for listening to my presentation.

References

Cawley, G.C. and Talbot, N.L.C. (2010) 'On over-fitting in model selection and subsequent selection bias in performance evaluation', *Journal of Machine Learning Research*, 11.

Glorot, X., Bordes, A. and Bengio, Y. (2010) 'Deep sparse rectifier neural networks', *Journal of Machine Learning Research*, 15, pp. 315–323.

Keras Documentation: MNIST digits classification dataset (no date) *Keras*. Available at: https://keras.io/api/datasets/mnist/ (Accessed: September 2026).

Kingma, D.P. and Ba, J.L. (2014) 'Adam: A method for stochastic optimization', *Conference paper at the 3rd International Conference for Learning Representations*, 1, pp. 1–15. doi:arxiv:1412.6980.

Lecun, Y. *et al.* (1998) 'Gradient-based learning applied to document recognition', *Proceedings of the IEEE*, 86(11), pp. 2278–2324. doi:10.1109/5.726791.

Srivastava, N. *et al.* (2014) 'Dropout: A simple way to prevent neural networks from overfitting', *Journal of Machine Learning Research*, 15, pp. 1929–1958.